

v1.6 — Document Management, Electronic Records & Archiving Specification

Absolutely. **v1.6** is an important specification for ILCMS because documents are effectively the core evidence and record objects around which the case-management system is built.

ILCMS — Icelandic Legal Case Management System

Technical Specification v1.6

Document Management, Electronic Records & Archiving Specification

Project: Icelandic Legal Case Management System (ILCMS)

Version: 1.6

Status: Technical Specification

Target Market: Iceland

Primary Language: Icelandic

Backend: Python / FastAPI

Database: PostgreSQL

Object Storage: S3-compatible private object storage

Deployment: Kubernetes / private cloud / on-premises / air-gapped capable

1. Purpose

This specification defines the architecture and implementation requirements for managing electronic documents and records within ILCMS.

The document subsystem shall support the complete lifecycle of legal case documents:

Capture

↓

Validation

↓

Security scanning

↓

Registration

↓

Classification

↓

Versioning

↓

OCR / text extraction

↓

Indexing



Use



Sharing / submission



Retention



Archiving / disposal

The design must support both ordinary working documents and documents that ultimately become part of an authoritative legal record.

2. Design Objectives

The document subsystem shall provide:

- secure document storage;
- document metadata;
- document classification;
- case association;
- version control;
- document integrity;
- OCR;
- full-text indexing;
- document preview;
- document download;
- document lifecycle management;
- retention policies;
- archival/export capabilities;
- legal hold;
- auditability;
- document access control;
- duplicate detection;
- malware scanning;

- immutable record handling.

The architecture must distinguish between:

working content and **authoritative records**.

This distinction is important because a document that is still being edited has different lifecycle requirements from a finalized court submission or received legal document.

3. Document vs Record

ILCMS shall distinguish between:

Document

A digital file managed by the system.

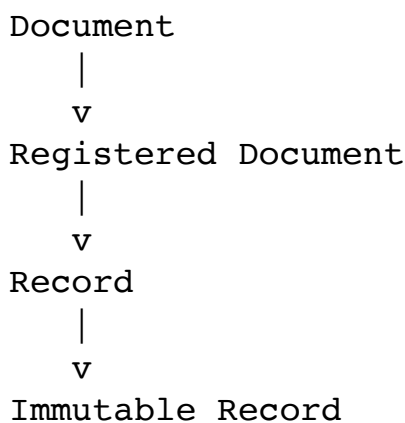
Examples:

- DOCX;
- PDF;
- XLSX;
- image;
- email;
- scanned paper document;
- text file.

Record

A document or document package that has been formally registered as part of the case record.

Conceptually:



Not every temporary working file needs to become a permanent record.

4. Record Lifecycle

A recommended lifecycle is:

DRAFT

↓

REGISTERED

↓

FINAL

↓

RETAINED

↓

ARCHIVED

Alternative states may include:

QUARANTINED

PROCESSING

REJECTED

SUPERSEDED

ON_HOLD

DISPOSED

The exact lifecycle shall be configurable.

5. Document Categories

ILCMS should support document categories such as:

INCOMING

OUTGOING

INTERNAL

COURT_SUBMISSION

COURT_DECISION

CONTRACT

EVIDENCE

CORRESPONDENCE

NOTE

REPORT

EMAIL

ATTACHMENT

SCAN

OTHER

These are machine-readable values.

The Icelandic user interface should provide localized labels.

6. Case Association

A document may be associated with:

- one primary case;
- optionally multiple related cases;
- a party;
- a task;
- a communication;
- a court submission;
- an event.

The primary case association should be explicit.

Example:

Document

- Primary Case
- Document Type
- Classification
- Author
- Sender
- Recipient
- Versions

7. Document Metadata

The document metadata model shall support at least:

document_id
case_id
organization_id
document_number
title
description
document_type
classification_id
status
created_at
received_at
document_date

registered_at
created_by
registered_by
sender
recipient
language
mime_type
file_size
checksum
current_version_id
confidentiality_level
retention_policy_id
Additional metadata should be extensible.

8. Icelandic Records Metadata

The system should support metadata relevant to Icelandic records-management practices.

For a case, this includes information such as:

- case identifier;
- case opening date;
- classification;
- parties;
- case title.

For documents, the system should support:

- received date;
- document date;
- classification;
- sender;
- recipient;
- subject/title;
- case identifier.

These fields should be first-class structured metadata rather than merely text embedded in the document.

9. Document Identifier

Every document receives an immutable UUID.

Example:

`document_id:`
`550e8400-e29b-41d4-a716-446655440000`
The UUID must never be reused.

Document identifiers should not expose:

- database sequence numbers;
- storage paths;
- internal server information.

10. Human-Readable Document Number

ILCMS may additionally generate a human-readable document number.

Example:

`M-2026-00125`
`DOC-000342`
The exact format should be configurable.

Human-readable identifiers must not replace the immutable UUID.

11. Document Title

A document title shall be stored independently of the filename.

Example:

`Filename:`
`attachment_20260902.pdf`

`Title:`
`Svar stefnda við kröfum stefnanda`
This is important for search, case lists and records management.

12. Filename

The original filename shall be preserved.

The system may additionally generate a normalized storage filename.

Example:

Original:

Krafa Jóns Árnasonar.pdf

Storage:

550e8400-e29b-41d4-a716-446655440000.pdf

The original filename should remain available as metadata.

13. Filename Security

User-provided filenames must never be directly used as filesystem paths.

The application must protect against:

- path traversal;
- directory traversal;
- special filesystem characters;
- excessively long filenames;
- control characters;
- Unicode normalization problems.

Storage keys should be generated by ILCMS.

14. Unicode

ILCMS must support Icelandic characters correctly, including:

Á É Í Ó Ú Ý Þ Æ Ð

á é í ó ú ý þ æ ð

Metadata, filenames and extracted text must use UTF-8.

Unicode normalization should be standardized.

15. Document Versions

Documents shall support immutable versions.

Conceptually:

Document

```
|  
+-- Version 1  
|  
+-- Version 2  
|  
+-- Version 3
```

Each version is immutable after creation.

A new version must create a new object.

16. Version Metadata

Each version should contain:

```
version_id  
document_id  
version_number  
storage_object_key  
file_size  
mime_type  
checksum  
created_at  
created_by  
processing_status  
Optional:
```

```
change_description  
source  
conversion_method
```

17. Current Version

The document record shall maintain a reference to the current version.

Example:

```
document.current_version_id
```

Historical versions remain available according to permissions and retention policy.

18. Version Immutability

Once a version has been finalized, its binary content must not be overwritten.

This provides:

- integrity;
- reproducibility;
- auditability;
- reliable historical reconstruction.

If a user edits a document:

```
Old Version
  |
  | remains unchanged
  |
+----> New Version
```

19. Checksums

Every stored document version should have a cryptographic checksum.

Recommended:

SHA-256

Example:

```
checksum_algorithm = SHA-256
checksum = <digest>
```

The checksum can be used to detect:

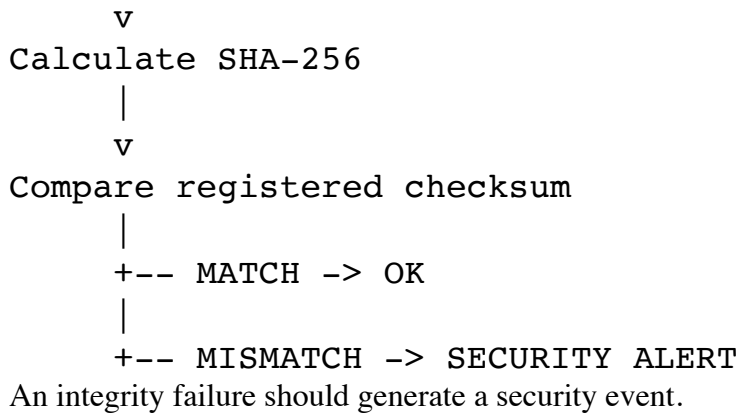
- corruption;
- accidental modification;
- duplicate files;
- storage inconsistencies.

20. Content Integrity

The system should periodically be capable of verifying stored document integrity.

Example:

```
Stored object
  |
```



21. Duplicate Detection

ILCMS should support duplicate detection using:

- checksum;
- file size;
- metadata;
- optionally content similarity.

Exact binary duplicates can be identified using SHA-256.

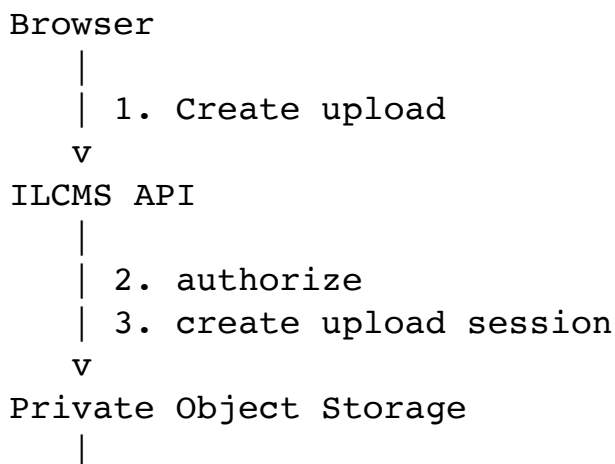
However, duplicates must not automatically be deleted.

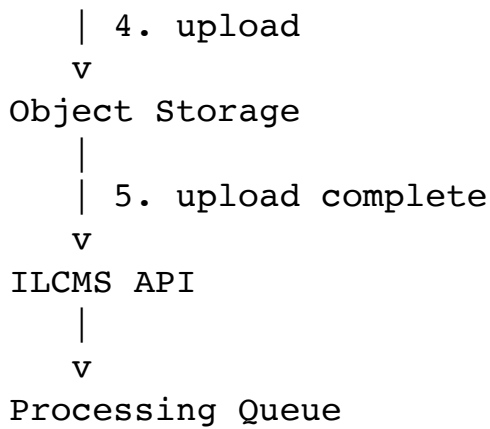
Two identical files may legitimately represent two different records.

22. Document Upload Architecture

Uploads should use the API/object-storage architecture defined in v1.4.

Recommended flow:





23. Upload Session

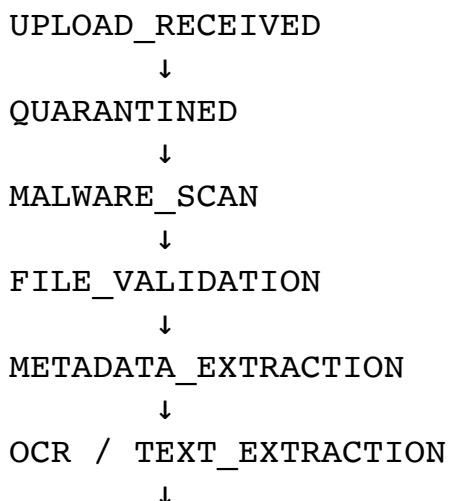
An upload session should contain:

upload_id
organization_id
user_id
case_id
expected_filename
expected_size
expected_mime_type
expires_at
status
Upload sessions must expire.

Unused temporary uploads should be cleaned automatically.

24. Upload Processing

After upload:



INDEXING

↓

AVAILABLE

Documents must not become generally downloadable before required security processing has completed.

25. Quarantine

New uploads should initially reside in a quarantine area.

Example:

`s3://ilcms/quarantine/<uuid>`

rather than immediately entering the permanent document area.

Only successfully processed files may move to normal document storage.

26. Malware Scanning

The document processing pipeline shall integrate with a malware scanner.

The system must record:

`scanner`
`scanner_version`
`scan_timestamp`
`result`

Possible results:

`CLEAN`
`INFECTED`
`ERROR`
`NOT_SCANNED`

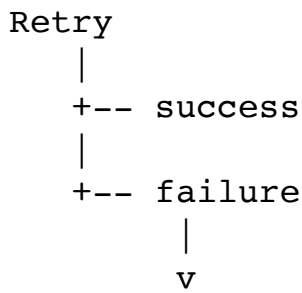
`INFECTED` documents must not be made available for download.

27. Processing Failures

Processing must be retryable.

Example:

`OCR failed`
|
v



Dead Letter Queue

The original document must not be lost merely because OCR or indexing fails.

28. Original File Preservation

The original uploaded binary must be preserved.

Derived files must not replace the original.

For example:

```
Original PDF
|
+-- OCR text
+-- preview PDF
+-- thumbnails
+-- extracted metadata
```

The original remains the authoritative source unless explicitly superseded by a later record/version.

29. OCR

OCR should be available for scanned documents and image-based PDFs.

The OCR subsystem should produce:

```
Extracted text
OCR confidence
OCR engine
OCR engine version
Processing timestamp
The original image/PDF must remain unchanged.
```

30. OCR Language

Icelandic OCR support is a significant requirement.

The system should support at least:

isl

and potentially:

eng

dan

nor

swe

where relevant.

The language configuration should be associated with the document or processing job.

31. OCR Quality

OCR should not silently replace original content.

If OCR confidence is poor, the document may be marked:

OCR_LOW_CONFIDENCE

The user should be able to inspect the original.

32. Text Extraction

Where documents already contain machine-readable text, direct extraction should be preferred over OCR.

Example:

PDF with text

|

v

Text extraction

rather than:

PDF with text

|

v

Rasterize

|

v

OCR

OCR should be used when necessary.

33. Search Index

Extracted text should be indexed separately from object storage.

Recommended architecture:

PostgreSQL

|

+-- metadata

Object Storage

|

+-- binaries

Search Index

|

+-- extracted text

+-- searchable metadata

The search index must never become an independent authorization boundary.

34. Search Authorization

Every search result must be filtered according to the current user's permissions.

The search system must not return:

- unauthorized case titles;
- unauthorized document titles;
- unauthorized snippets;
- unauthorized extracted text.

Authorization must be enforced before results are returned.

35. Searchable Metadata

Search should support fields such as:

case_number

case_title

document_number

document_title

document_type

classification
sender
recipient
document_date
received_date
filename
extracted_text

36. Icelandic Search

The search subsystem should support Icelandic text effectively.

Consider:

- Unicode normalization;
- Icelandic tokenization;
- stemming/lemmatization;
- diacritics;
- compound words;
- exact phrase matching.

The search engine should be evaluated using real Icelandic legal documents.

37. Document Preview

The frontend may provide previews for supported formats.

Typical formats:

PDF

JPEG

PNG

TXT

Preview generation must occur in a sandboxed processing environment.

Office documents should not be rendered by trusting embedded active content.

38. PDF Preview

PDF preview generation should be isolated from the main API process where practical.

This reduces the impact of vulnerabilities in document-processing libraries.

39. Office Documents

Office documents such as:

DOCX

XLSX

PPTX

must be treated as untrusted input.

The system should not execute:

- macros;
- embedded programs;
- external active content.

If conversion to PDF is supported, conversion should occur in an isolated worker.

40. Email Documents

ILCMS should support email as a document source.

An email record may contain:

From

To

CC

BCC

Subject

Date

Message-ID

Body

Attachments

The original email format should be preserved where possible.

41. Email Attachments

Attachments should be separately registered when appropriate.

Example:

Email

```
|
+-- Attachment A
+-- Attachment B
```

The relationship between the attachment and original email must remain traceable.

42. Electronic Court Documents

ILCMS shall support documents prepared for submission to Icelandic courts.

The system should support:

- descriptive filenames;
- document dates;
- searchable PDFs where applicable;
- separate electronic files;
- original photographs/images;
- document lists;
- structured submission packages.

The system must not automatically assume that every court has identical technical submission requirements.

Court-specific rules shall therefore be configurable.

43. Court Submission Package

A court submission should be modeled separately from individual documents.

Example:

```
Court Submission
|
+-- Main pleading
+-- Document list
+-- Exhibit 1
+-- Exhibit 2
+-- Exhibit 3
```

This provides traceability between the submitted package and its constituent files.

44. Submission Manifest

A submission package should optionally contain a manifest:

Submission ID

Case number

Submission date

Submitting party

Document sequence

Document titles

Checksums

The manifest provides integrity and reproducibility.

45. Document Ordering

Documents may have an explicit sequence number within a case or submission.

Example:

01 — Stefna

02 — Fylgiskjal A

03 — Fylgiskjal B

04 — Bréf

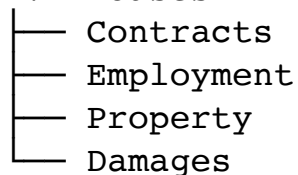
Sequence should be stored as structured metadata, not inferred from filenames.

46. Document Classification

Documents shall support classification using a configurable classification tree.

Example:

Civil cases



The exact classification system shall be configurable for each organization.

47. Málalykill Support

ILCMS should support the concept of a **málalykill** as a structured document classification mechanism.

The classification should be:

- hierarchical;

- versioned;
- organization-specific where required;
- searchable;
- auditable.

Changing a classification scheme must not destroy historical classification information.

48. Classification Versioning

If a classification code changes:

Old:

4.2.1

New:

5.1.3

historical records must retain the classification that applied when they were registered.

Migration mappings may additionally be stored.

49. Records Registration

A document becomes a registered record when required by the organization's records-management policy.

Registration should capture:

registered_at
registered_by
classification
case_id
document_date
received_date
sender
recipient
title
record_status

Registration must be auditable.

50. Incoming Documents

Incoming documents should support:

received_at
sender
recipient
received_channel
original_filename
document_date
case_id
classification
Possible channels:

EMAIL
PORTAL
PHYSICAL_SCAN
COURT
POST
MANUAL_UPLOAD
INTEGRATION
OTHER

51. Outgoing Documents

Outgoing documents should support:

created_at
sender
recipient
sent_at
delivery_method
document_date
case_id
classification
Delivery methods might include:

EMAIL
COURT_PORTAL
POST
SECURE_PORTAL
HAND_DELIVERY
OTHER

52. Document Status

Recommended document statuses:

DRAFT
REGISTERED
FINAL
SUPERSEDED
ARCHIVED
ON_HOLD
DISPOSED

Processing states should remain separate from business/record status.

For example:

```
record_status = FINAL  
processing_status = INDEXED
```

53. Legal Hold

ILCMS shall support legal holds.

A legal hold prevents normal disposal of affected records.

Example:

```
Retention policy  
    |  
    v  
Eligible for disposal  
    |  
    v  
LEGAL_HOLD = TRUE  
    |  
    v  
DISPOSAL_BLOCKED
```

Legal holds should be:

- explicit;
- auditable;
- associated with a reason;
- associated with the creator;
- optionally associated with an expiry/review date.

54. Retention Policies

Retention policies shall be configurable.

A policy may define:

```
policy_id
name
description
classification
retention_period
trigger_event
disposal_action
legal_hold_behavior
```

Example:

Retention:

10 years after case closure

The example is illustrative only; actual periods must be configured according to applicable requirements and organizational policy.

55. Retention Trigger

Retention should be based on a defined event rather than simply document creation.

Possible triggers:

```
CASE_CLOSED
RECORD_REGISTERED
CONTRACT_EXPIRED
COURT_PROCEEDING_COMPLETED
CUSTOM_EVENT
```

56. Disposal Eligibility

When a record reaches the end of its retention period:

```
Retention expired
|
v
Check legal hold
|
+-- YES --> HOLD
|
+-- NO --> Review
|
v
Dispose
```

The system must not silently delete records.

57. Disposal Approval

For high-value records, disposal should require explicit authorization.

The system should support:

ELIGIBLE

↓

PENDING_REVIEW

↓

APPROVED

↓

DISPOSED

Disposal events must be audited.

58. Disposal Audit

Even when a document is disposed of, the system should preserve an appropriate audit record stating:

document_id

case_id

classification

disposal_date

disposal_policy

authorized_by

reason

The audit record must itself follow applicable retention requirements.

59. Archiving

Archiving should be treated as a controlled lifecycle transition rather than simply moving files to another directory.

Active Record

|

v

Archive Candidate

|

v

Archive Package

|

60. Archival Package

ILCMS should be capable of producing a self-contained archival package containing:

Metadata
Documents
Relationships
Checksums
Classification
Case information
Version information
Audit information where appropriate
Manifest

A standardized archival format should be selected during implementation based on the target archival environment.

61. Archival Manifest

Example:

```
{
  "package_id": "uuid",
  "created_at": "2026-09-02T08:00:00Z",
  "case_id": "uuid",
  "case_number": "M-2026-00125",
  "documents": [
    {
      "document_id": "uuid",
      "version_id": "uuid",
      "filename": "stefna.pdf",
      "checksum": "sha256:..."
    }
  ]
}
```

The manifest provides integrity verification.

62. Archive Verification

After archival export/import, the system should verify:

Package readable?
Metadata complete?
Documents present?
Checksums match?
Relationships preserved?
Failed verification must prevent the archive from being marked successful.

63. Preservation Formats

For long-term preservation, ILCMS should distinguish between:

Original format

and:

Preservation/access format

For example:

Original:
DOCX

Derived:
PDF/A

The original should normally remain available where policy requires it.

The exact preservation formats should be determined according to the target archival environment.

64. PDF/A

PDF/A should be supported as a possible preservation format for appropriate documents.

However, conversion must not automatically destroy or replace the original document.

Original DOCX
|
+----> Preservation PDF/A

Both may be retained where required.

65. Photographs and Images

Original photographs should be preserved without unnecessary conversion.

The system may generate:

Original JPEG

|

+-- Thumbnail

+-- Preview

but the original must remain intact.

Metadata such as EXIF should be handled according to configured privacy and records-management requirements.

66. Document Metadata Extraction

The system may extract:

- author;
- creation date;
- modification date;
- application;
- page count;
- image dimensions;
- PDF properties.

Extracted metadata must be distinguished from authoritative registered metadata.

For example:

```
metadata.source = EXTRACTED
```

versus:

```
metadata.source = REGISTERED
```

67. Metadata Precedence

Where extracted metadata conflicts with registered metadata:

Authoritative registered metadata

>

Extracted metadata

>

Filename-derived metadata

The system must not silently overwrite authoritative metadata using metadata embedded in uploaded files.

68. Document Relationships

Documents may have relationships such as:

SUPERSEDES
SUPERSEDED_BY
ATTACHMENT_OF
RELATED_TO
REPLY_TO
DERIVED_FROM
TRANSLATION_OF

This enables reconstruction of document history.

69. Translations

A translated document should be linked to the original.

Example:

Original:
Icelandic

|
+----- TRANSLATION_OF ----->

English translation

The translation should not replace the original.

Language must be stored explicitly.

70. Confidentiality Levels

ILCMS should support configurable confidentiality levels.

Example:

NORMAL
CONFIDENTIAL
HIGHLY_CONFIDENTIAL
RESTRICTED

These are application security labels, not legal classifications unless explicitly mapped to an applicable legal framework.

71. Access Restrictions

Documents may have additional access restrictions beyond case membership.

For example:

```
Case access
+
Document restriction
=
Effective access
```

The most restrictive applicable rule should normally prevail.

72. Document Sharing

Sharing a document outside the normal case-access group must be explicit.

A sharing operation should record:

```
document
recipient
permissions
creator
created_at
expiration
```

Temporary external sharing should use short-lived access mechanisms.

73. External Sharing

External document sharing should preferably use a controlled portal rather than sending permanent object-storage URLs.

Example:

```
ILCMS
|
v
Secure Share
|
+-- Expiration
+-- Authentication
+-- Download audit
+-- Revocation
```

74. Download Auditing

Document downloads should generate audit events where appropriate.

The system should record:

```
user
document
version
timestamp
organization
request_id
result
```

This is particularly important for sensitive legal material.

75. Printing

Printing is difficult to control technically.

Where required, ILCMS may record:

```
PRINT_REQUESTED
```

but must not claim that a physical printout was actually produced unless the environment provides reliable printer integration.

76. Watermarking

Optional watermarking may be supported for generated copies.

Example:

```
CONFIDENTIAL
```

```
ILCMS
```

```
Generated for: User
```

```
2026-09-02
```

Watermarks must not alter the authoritative original.

77. Redaction

Future versions may support document redaction.

Redaction must be destructive to the exported copy or otherwise cryptographically/technically safe.

Simply drawing a black rectangle over text is **not** sufficient.

A redacted PDF must not retain the hidden original text underneath the redaction.

78. Document Conversion

Conversions must be performed in isolated worker environments.

Potential conversions:

DOCX -> PDF

ODT -> PDF

PDF -> PDF/A

Image -> PDF

Conversion results should be treated as derived artifacts.

79. Document Processing Security

Document parsers and converters are high-risk attack surfaces.

Processing workers should use:

- restricted containers;
- no unnecessary network access;
- non-root execution;
- resource limits;
- timeouts;
- temporary isolated storage.

80. Archive Storage

Archived records should ideally be stored separately from active working documents.

For example:

Active Storage

|
v

Archive Storage

Archive storage may have stronger:

- immutability;

- retention;
- access controls;
- backup requirements.

81. Object Storage Layout

A logical storage layout may be:

```
organizations/  
  <organization-id>/  
    cases/  
      <case-id>/  
        documents/  
          <document-id>/  
            versions/  
              <version-id>
```

However, this is a logical namespace.

Applications should not depend on the physical layout.

Storage keys must be generated centrally.

82. Object Versioning

Object storage versioning should be enabled where supported.

This provides another layer of protection against:

- accidental overwrite;
- deletion;
- corruption;
- ransomware.

Application-level document versioning remains authoritative.

83. Object Deletion

Document deletion should normally mean a lifecycle/state transition rather than immediate physical deletion.

For example:

ACTIVE

↓

DELETED / DISPOSED

↓

Physical deletion

The second step is governed by retention and storage policy.

84. Document Locking

For collaborative editing, ILCMS may support document locking.

Possible states:

UNLOCKED

LOCKED

Lock metadata:

locked_by

locked_at

expires_at

Locks must automatically expire to prevent permanent deadlocks.

85. Optimistic Concurrency

Where documents or metadata can be edited concurrently, ILCMS should use optimistic concurrency.

Possible mechanisms:

- version numbers;
- ETags;
- If-Match.

Example:

If-Match: "document-version-7"

If the document changed meanwhile:

409 Conflict

86. Document API

The v1.4 API should support at least:

```
POST    /api/v1/documents
GET     /api/v1/documents/{id}
PATCH  /api/v1/documents/{id}
DELETE  /api/v1/documents/{id}

POST    /api/v1/cases/{case_id}/documents
GET     /api/v1/cases/{case_id}/documents

POST    /api/v1/documents/{id}/versions
GET     /api/v1/documents/{id}/versions

GET     /api/v1/documents/{id}/download
GET     /api/v1/documents/{id}/processing

POST    /api/v1/documents/{id}/register
POST    /api/v1/documents/{id}/finalize
POST    /api/v1/documents/{id}/hold
Exact endpoint semantics remain governed by v1.4.
```

87. Document Processing API

Processing status should expose states such as:

```
UPLOAD
MALWARE_SCAN
VALIDATION
METADATA
OCR
TEXT_EXTRACTION
INDEXING
Example:
```

```
{
  "document_id": "uuid",
  "status": "PROCESSING",
  "steps": [
    {
      "name": "malware_scan",
      "status": "COMPLETED"
    },
    {
      "name": "ocr",
      "status": "RUNNING"
    }
  ],
}
```

```
{
  {
    "name": "indexing",
    "status": "PENDING"
  }
}
```

88. Background Processing

Long-running document operations must run asynchronously.

Examples:

- OCR;
- conversion;
- indexing;
- virus scanning;
- archive generation;
- bulk import;
- redaction;
- document export.

These operations should use the worker/message architecture defined in v1.1.

89. Idempotency

Processing jobs should be idempotent.

If the same job executes twice:

OCR(document_version=7)
the result must not corrupt the document or create duplicate authoritative records.

90. Document Events

The document subsystem should emit domain events such as:

DocumentUploaded
DocumentScanned
DocumentValidated

DocumentRegistered
DocumentVersionCreated
DocumentFinalized
DocumentDownloaded
DocumentShared
DocumentPlacedOnHold
DocumentArchived
DocumentDisposed
These events may be consumed by:

- audit;
- notifications;
- search;
- workflow;
- integrations.

91. Event Security

Domain events must not contain unnecessary document contents.

Prefer:

```
{  
  "event": "DocumentUploaded",  
  "document_id": "uuid",  
  "case_id": "uuid",  
  "organization_id": "uuid"  
}
```

rather than placing the entire document in the message.

92. Document Import

ILCMS should support controlled bulk document import.

Possible sources:

- filesystem;
- ZIP archive;
- email;
- legacy case-management system;

- API;
- cloud/object storage.

Imports must go through the same security controls as normal uploads.

93. Import Mapping

Legacy migration should support mapping:

Old Case ID

↓

ILCMS Case ID

Old Document ID

↓

ILCMS Document ID

Original identifiers should be retained as migration metadata where useful.

94. Bulk Import Validation

Before committing a bulk import:

Validate files

Validate metadata

Validate cases

Validate classifications

Validate relationships

Validate duplicates

Validate permissions

A dry-run mode should be provided.

95. Document List

Each case should have a document list showing at least:

Document number

Title

Document type

Date

Sender/recipient

Classification

Status

Version

Users should be able to sort and filter.

96. Document List for Court Submissions

A court submission document list should support explicit ordering and titles.

Example:

1. Stefna
2. Samningur dags. 12. janúar 2026
3. Tölvupóstsamskipti
4. Ljósmyndir

The generated list should be stored as a document/record where appropriate.

97. Case Timeline Integration

Documents should be capable of appearing in the case timeline.

Example:

```
02.09.2026 Document received
02.09.2026 Lawyer reviewed document
03.09.2026 Reply sent
05.09.2026 Court submission created
```

Timeline events should reference document IDs rather than duplicate document metadata unnecessarily.

98. Document Retention and Case Retention

Document retention should normally be compatible with the case lifecycle.

However, the system must support exceptions.

For example:

Case closed

```
|
+-- Most records -> normal retention
|
+-- Record under legal hold -> retained
|
+-- Special record -> different policy
```

99. Archival Compatibility

The system should be designed so that a complete case can eventually be exported without requiring the original ILCMS software.

An archival package should contain sufficient:

- metadata;
- files;
- relationships;
- identifiers;
- classification information;
- integrity information

to reconstruct the record independently.

100. Long-Term Accessibility

Long-term preservation should consider:

- obsolete formats;
- unsupported software;
- corrupted media;
- cryptographic algorithm changes;
- encoding changes.

The archive subsystem should support periodic preservation review.

101. Audit Requirements

Document audit events should cover at least:

Upload

View

Download

Create version

Register

Finalize

Share
Change metadata
Change classification
Change access
Delete
Restore
Legal hold
Release hold
Archive
Dispose
Export

Access to particularly sensitive documents may require additional logging.

102. Document Access Reports

Authorized administrators/auditors should be able to determine:

Who accessed this document?

When?

From which organization?

Which version?

What action?

Was it successful?

This capability should use the audit subsystem rather than application logs.

103. Security and Privacy

The document subsystem must assume that every uploaded file may contain sensitive information.

Therefore:

- documents are private by default;
- storage is private;
- access is authorized;
- downloads are controlled;
- logs are minimized;
- derived text is protected;
- search indexes are protected;
- backups are protected.

104. Search Index Security

The search index may contain substantially the same confidential information as the documents themselves.

Therefore it must have equivalent or stronger access controls.

A compromised search index must not become an easy method of extracting document contents.

105. OCR Security

OCR output must be treated as confidential case information.

For example:

PDF

↓

OCR

↓

Extracted text

The extracted text must receive the same organization/case security boundary as the source document.

106. Temporary Files

Document-processing workers may generate temporary files.

Temporary data must:

- reside in isolated storage;
- have restrictive permissions;
- be automatically deleted;
- not contain unnecessary persistent copies.

107. Storage Cleanup

The system should periodically clean:

- expired upload sessions;
- failed temporary uploads;

- abandoned processing files;
- expired exports;
- expired sharing links;
- temporary conversion artifacts.

Cleanup must respect legal holds and retention policies.

108. Document Recovery

A deleted or accidentally modified document should be recoverable where permitted by retention/storage policy.

Recovery must preserve:

- original identifier;
- version history;
- audit history.

Restoration should generate an audit event.

109. Disaster Recovery

Document recovery must be tested independently from database recovery.

A valid disaster recovery test should demonstrate:

Database restored

+

Object storage restored

+

Search rebuilt/restored

+

Document relationships intact

A restored database without its document objects is not a successful ILCMS recovery.

110. Database/Object Storage Consistency

The system must account for the fact that PostgreSQL and object storage are separate systems.

Example failure:

DB transaction succeeds

|
X

Object upload fails

or:

Object upload succeeds

|
X

DB transaction fails

The upload state machine and reconciliation jobs must detect and resolve these inconsistencies.

111. Storage Reconciliation

A scheduled reconciliation job should identify:

DB record without object

Object without DB record

Checksum mismatch

Orphaned version

Incomplete upload

Expired temporary object

Discrepancies should generate operational/security alerts where appropriate.

112. Document Integrity Verification

Periodic integrity verification should calculate checksums for selected or all archived documents.

For example:

Archive verification job

|
+-- checksum match -> OK

|
+-- mismatch -> CRITICAL ALERT

The verification schedule should be configurable.

113. Archival Export Security

Archive exports can contain entire cases.

Therefore:

- explicit permission is required;
- exports are audited;
- export packages are encrypted;
- temporary links expire;
- downloaded packages should be protected;
- export generation occurs asynchronously.

114. Records Management Configuration

Organizations should be able to configure:

Classification schemes

Document types

Retention policies

Confidentiality levels

Registration requirements

Archive rules

Disposal workflow

Court submission profiles

System administrators should not hard-code these values into application logic.

115. Icelandic Court Profiles

The architecture should allow court-specific profiles.

Example:

Court profile

```
|
+-- Required filename convention
+-- Allowed formats
+-- Document ordering
+-- Document list requirements
+-- Submission metadata
+-- Maximum file size
+-- Delivery mechanism
```

This makes it possible to adapt the system when court procedures change.

116. Court Rules Must Be Configurable

ILCMS must not encode a permanent assumption such as:

"All court documents must be PDF."

Instead:

Court

|

v

Submission Profile

|

+-- allowed formats

+-- naming

+-- metadata

+-- package structure

This protects the architecture against changes in court rules.

117. Records Authority Integration

If an organization must transfer records to an archival authority, ILCMS should support controlled export.

The integration should be adapter-based:

ILCMS Archive

|

v

Archive Adapter

|

+-- File export

+-- Metadata export

+-- Manifest

+-- Validation

|

v

Target Archive

No specific external archival API should be assumed at this stage.

118. Archive Transfer Audit

An archival transfer should record:

transfer_id

case/range

records

created_at
initiated_by
approved_by
destination
package_checksum
transfer_status

119. Archive Transfer States

Recommended:

PREPARING
VALIDATING
READY
APPROVED
TRANSFERRED
VERIFIED
FAILED

A record should not be considered successfully archived merely because an export file was generated.

120. Document Management Data Model

The v1.3 database should support at least:

documents
document_versions
document_access
document_relationships
document_processing_jobs
document_metadata
document_classifications
document_registrations
document_retention
legal_holds
archive_packages
archive_transfers

Potentially:

document_shares
document_exports
document_integrity_checks

121. Suggested Document Tables

Conceptually:

documents

id

organization_id

case_id

document_number

title

document_type

classification_id

status

created_at

created_by

current_version_id

registered_at

registered_by

retention_policy_id

legal_hold

document_versions

id

document_id

version_number

object_key

filename

mime_type

size

sha256

created_at

created_by

finalized_at

document_processing

id

document_version_id

job_type

status

started_at

completed_at

worker

error_code

122. Storage Separation

ILCMS should logically separate:

Quarantine

Active

Archive

Temporary

Exports

Example:

Object Storage

|

+-- quarantine/

+-- active/

+-- archive/

+-- temporary/

+-- exports/

Each area should have different lifecycle and access policies.

123. Retention of Derived Data

Retention must account for derived data.

If a document is retained, associated:

- OCR text;
- thumbnails;
- search index entries;
- metadata;
- derived PDF/A

may also need retention.

When a record is disposed of, derived copies must not remain indefinitely.

124. Secure Disposal

Secure disposal should address:

Original object

Versions

OCR text

Search index

Thumbnails

Previews

Exports

Temporary files

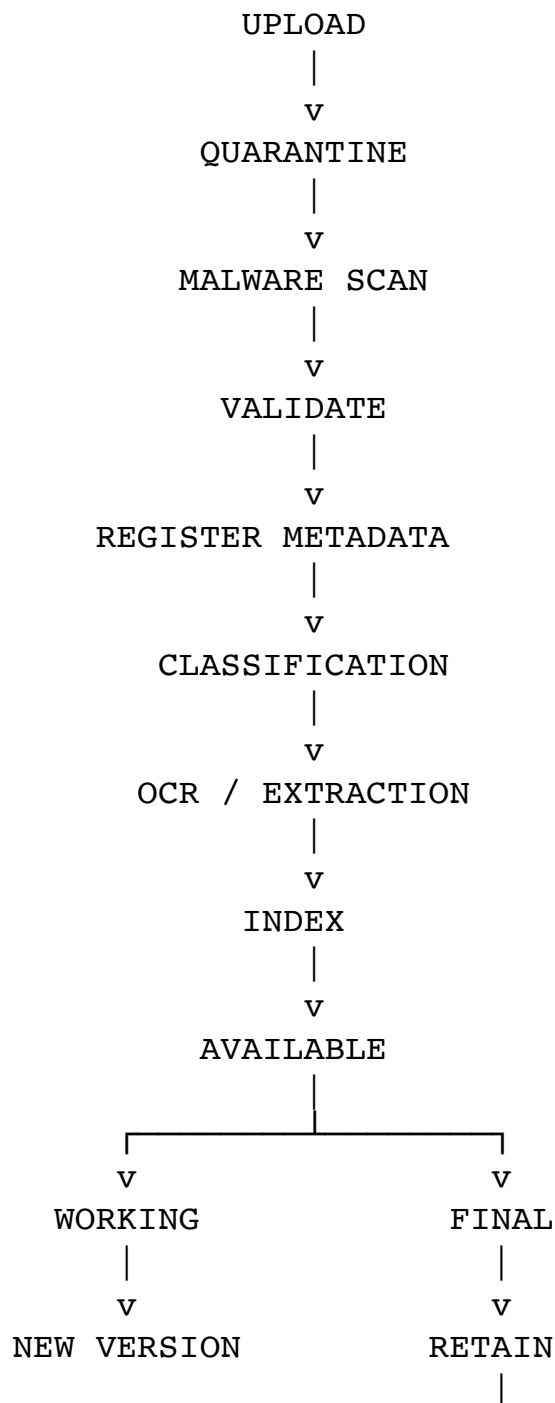
Caches

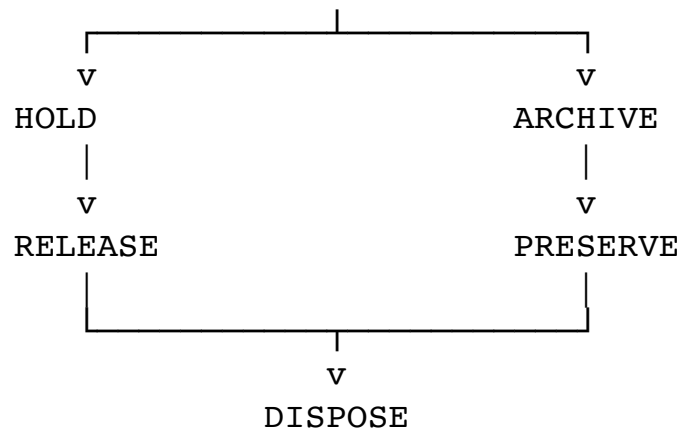
Backups

Physical destruction of backup media is a separate infrastructure concern.

125. Document Lifecycle Summary

The complete lifecycle is:





126. MVP Scope

The initial ILCMS document-management implementation should prioritize:

Required

- document upload;
- private object storage;
- case association;
- metadata;
- document types;
- classification;
- versioning;
- SHA-256 checksums;
- malware scanning;
- OCR;
- text extraction;
- search indexing;
- access control;
- download authorization;
- audit events;
- retention metadata;
- legal hold;

- basic export.

Later

- advanced archival packages;
- court-specific automated submission;
- automated redaction;
- advanced document similarity;
- digital signatures;
- advanced preservation workflows;
- archival authority integrations.

127. Definition of Done

v1.6 is complete when:

1. Documents can be uploaded securely.
2. Files enter quarantine before normal availability.
3. Malware scanning is mandatory.
4. Original files are preserved.
5. Every version has an immutable identifier.
6. SHA-256 integrity hashes are stored.
7. Documents can be associated with cases.
8. Required metadata can be registered.
9. Classification is supported.
10. Icelandic characters are fully supported.
11. OCR supports Icelandic documents.
12. Extracted text is searchable.
13. Search respects authorization.
14. Document downloads require authorization.
15. Object storage remains private.

16. Document versions cannot be overwritten.
17. Legal holds prevent disposal.
18. Retention policies are configurable.
19. Disposal is auditable.
20. Archive packages can be generated.
21. Archive packages contain integrity manifests.
22. Database/object-storage inconsistencies can be detected.
23. Document processing is asynchronous.
24. Processing failures are retryable.
25. Audit events cover sensitive document operations.
26. Backups include both metadata and document objects.
27. Disaster recovery has been tested.
28. Court submission profiles are configurable.
29. The system can operate without Internet access in an air-gapped deployment.
30. Applicable Icelandic records-management and court requirements have been validated against the actual deployment context before production.

128. Architectural Principle

The most important principle of the ILCMS document subsystem is:

The database stores authoritative document metadata and relationships; private object storage stores immutable document content; the search engine stores derived searchable information; and the audit system records what happened to the document. None of these components should independently bypass the authorization model.

This separation provides a strong foundation for:

- legal case management;
- electronic records management;
- document preservation;
- court submissions;
- future archival transfer;
- secure air-gapped deployment.

129. Relationship to Previous Specifications

The document subsystem integrates with the previous specifications as follows:

v1.1 DevOps

- |
- +-- Object storage
- +-- Workers
- +-- Message queues
- +-- Backup
- +-- Kubernetes

v1.2 Backend

- |
- +-- Document domain services
- +-- Processing services
- +-- Repository layer

v1.3 Database

- |
- +-- Document metadata
- +-- Versions
- +-- Relationships
- +-- Retention

v1.4 REST API

- |
- +-- Upload
- +-- Download
- +-- Metadata
- +-- Processing
- +-- Archive

v1.5 Security

- |
- +-- Authentication
- +-- Authorization
- +-- Tenant isolation
- +-- Audit
- +-- Encryption

v1.6 Documents

- |
- +-- Records

- +-- OCR
- +-- Classification
- +-- Retention
- +-- Archiving

This creates the foundation for the next major layer of the application: **case workflow and legal-process management**.

With v1.6 in place, the architecture now has a reasonably complete **secure document/records foundation**. The next specification I would recommend is **v1.7 — Case Management, Workflow, Tasks, Deadlines & Legal Process Specification**, covering the actual day-to-day work of lawyers and case managers: case states, assignments, tasks, deadlines, hearings, court events, correspondence, reminders, workflows, escalation, and the relationship between all of these objects.